

practical-6

Aim: Implementation of chain matrix multiplication using dynamic programming.

Matrix Chain multiplication

Algorithm:

Algorithm:

Input: Numbers of matrices n
Array $r[0 \dots n]$ containing the dimensions of the matrices

Output: minimum no. of scalar multiplication required to multiply the given chain of matrices.

Step 1: Start

Step 2: Read the no. of matrices n

Step 3: Read the dimension array $r[0 \dots n]$
where matrix x_i has dimension $r[i-1] \times r[i]$.

Step 4: Initialize the diagonal values.
 $C[i][i] = 0$

Step 5: Consider chains of increasing length $width$.

Step 6: For each pair of increasing length
Set $min = 32767$.

Step 7: Calculate the multiplication cost:-
 $C[i][k] + C[k+1][j] + r[i-1] \times r[k] \times r[j]$

Step 8: If $q < min$, update min and store the splitting position k .

Step 9: Store the minimum cost in $C[i][j] = min$.

Step 10: After calculating all possible chains, print $C[0][n]$.

Step 11: Stop

Program:

```
#include <iostream>
using namespace std;

int main()
{
    int n, i, j, k, d, q, min;
    int p[20], c[20][20], k_table[20][20];

    cout << "Enter number of matrices: ";
    cin >> n;

    cout << "Enter the dimensions:" << endl;
    for (i = 0; i <= n; i++)
    {
        cin >> p[i];
    }

    for (i = 1; i <= n; i++)
    {
        c[i][i] = 0;
    }

    for (d = 1; d <= n - 1; d++)
    {
        for (i = 1; i < n - d; i++)
        {
            j = i + d;
            min = 32767;

            for (k = 1; k <= j - 1; k++)
            {
                q = c[i][k] + c[k + 1][j] + p[i - 1] * p[k] * p[j];

                if (q < min)
                {
                    min = q;
                    k_table[i][j] = k;
                }
            }
        }
    }
}
```



```
        c[i][j] = min;
    }
}

cout << "\nMinimum number of multiplications = " << c[1][n] << endl;

return 0;
}
```

Output:

```
Output

Enter number of matrices: 4
Enter the dimensions:
10 20
20 30
30 40

Minimum number of multiplications = 19000

=== Code Execution Successful ===
```

Time Complexity:

Best Case:

Time complexity:

* Best case:

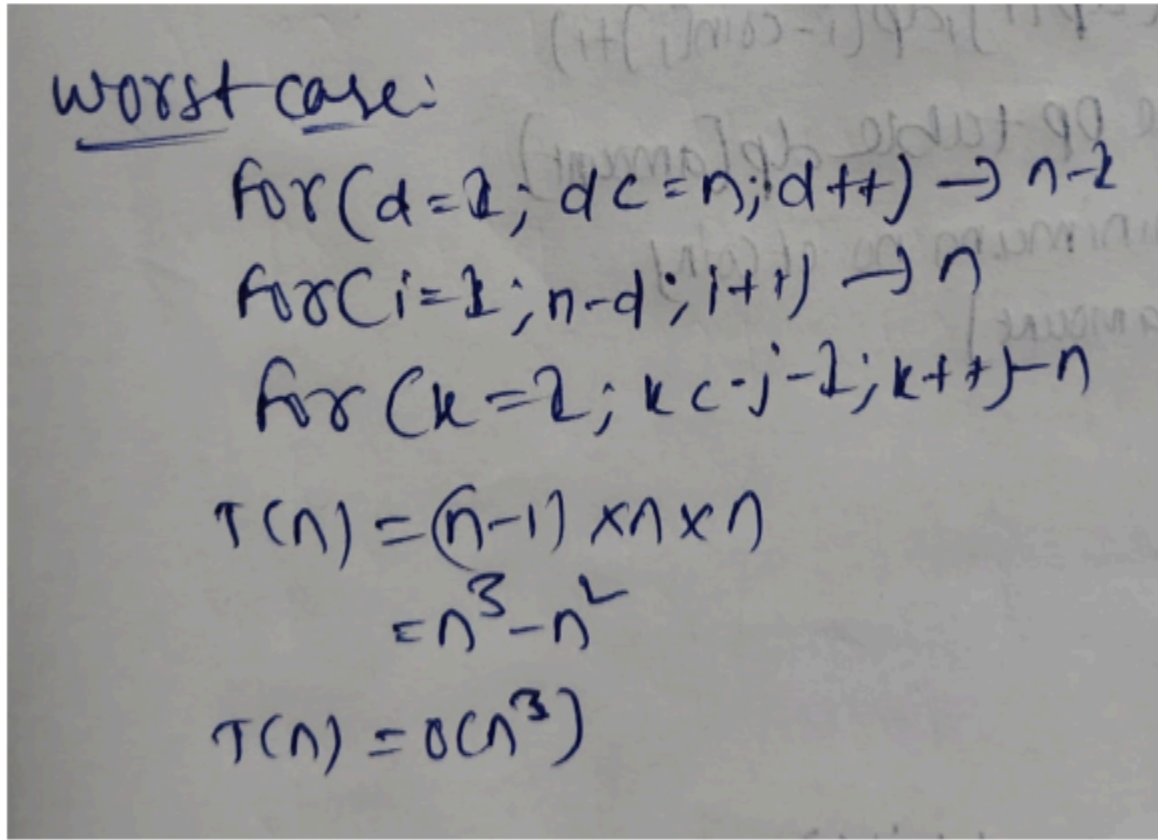
$$\text{for}(d=1; d \leq n-1; d++) \rightarrow n-1$$
$$\text{for}(i=1; i \leq n-d; i++) \rightarrow n$$
$$\text{for}(k=1; k \leq j-1; k++) \rightarrow n$$
$$T(n) = (n-1) \times n \times n$$
$$= n^3 - n^2$$
$$T(n) = O(n^3)$$

Average Case:

Average case:

$$\text{for}(d=1; d \leq n-1; d++) \rightarrow n-1$$
$$\text{for}(i=1; i \leq n-d; i++) \rightarrow n$$
$$\text{for}(k=1; k \leq j-1; k++) \rightarrow n$$
$$T(n) = (n-1) \times n \times n$$
$$= n^3 - n^2$$
$$T(n) = O(n^3)$$

Worst Case:



worst case:

$$\text{for}(d=2; d \leq n; d++) \rightarrow n-2$$
$$\text{for}(i=1; i \leq n-d; i++) \rightarrow n$$
$$\text{for}(k=1; k \leq i-1; k++) \rightarrow n$$
$$T(n) = (n-1) \times n \times n$$
$$= n^3 - n^2$$
$$T(n) = O(n^3)$$

Conclusion:

The implementation and time analysis of Matrix Chain Multiplication using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different ways of splitting the matrix chain to find the minimum number of scalar multiplications. It has a time complexity of $O(n^3)$ in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Matrix Chain Multiplication problem.